

Linux Application Development - Day 1

Build Tools, Compilers & Libraries

LFD401: Developing Applications for Linux

Welcome to Day 1

Course Overview

This intensive 5-day course covers:

- **Day 1:** Build tools, compilers, libraries
- **Day 2:** Debugging, system calls, memory management
- **Day 3:** File systems, I/O operations
- **Day 4:** Process management, IPC, signals
- **Day 5:** Threading, networking, sockets

Today's Agenda

What You'll Master Today

1. **Compilation pipeline** - From source to executable
2. **GCC toolchain** - Compiler options and optimization
3. **Static and shared libraries** - Creation and usage
4. **Build systems** - Make and Makefiles
5. **Version control** - Git workflows for C projects
6. **Development environments** - IDEs and debugging integration

Lab time: ~4 hours | **Slides:** ~50

Section 1: The Compilation Process

Understanding Compilation

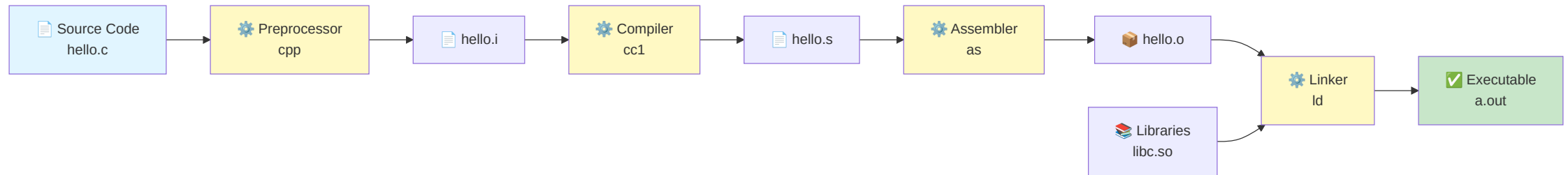
The Four Stages

Every C program goes through these stages:

1. **Preprocessing** - Handle directives (`#include` , `#define`)
2. **Compilation** - Convert C to assembly
3. **Assembly** - Convert assembly to machine code
4. **Linking** - Combine object files and libraries

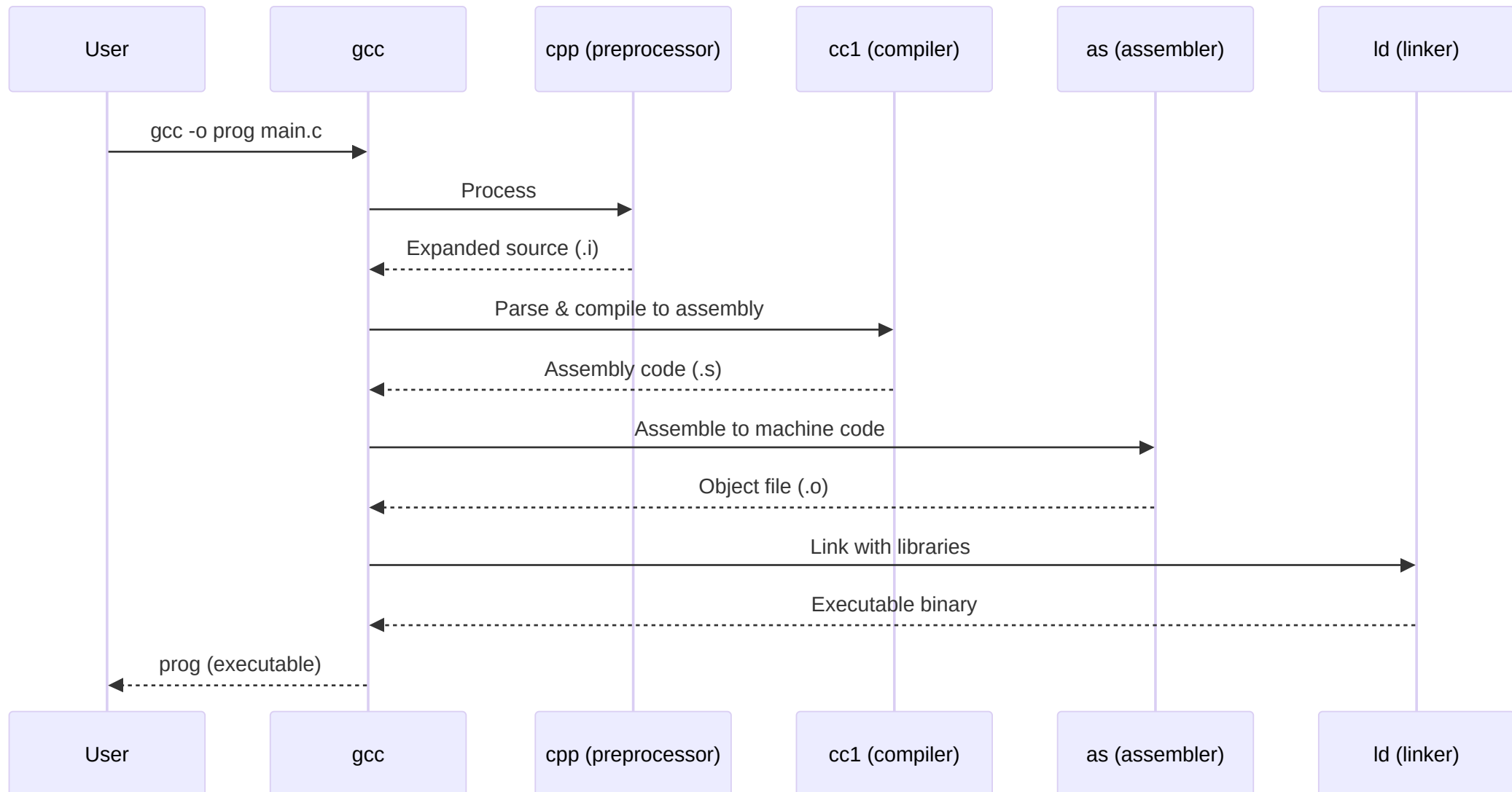
Key insight: `gcc` is a driver that orchestrates these stages

The Compilation Pipeline



Each stage can be invoked separately for debugging

GCC Stage by Stage



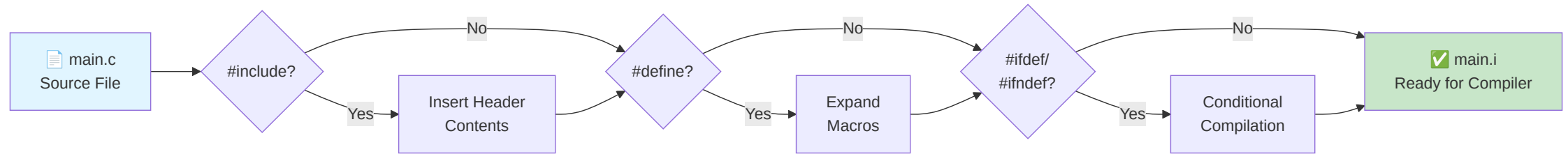
Stage 1: Preprocessing

What The Preprocessor Does

- Strips comments
- Expands `#include` directives (inserts file contents)
- Expands `#define` macros
- Evaluates conditional compilation (`#ifdef` , `#ifndef`)
- Produces `.i` file (preprocessed source)

Command: `gcc -E source.c -o source.i`

Preprocessor Flow



Preprocessor Example

```
#include <stdio.h>
#define MAX_SIZE 100
#define SQUARE(x) ((x) * (x))

#ifdef DEBUG
    #define LOG(msg) printf("DEBUG: %s\n", msg)
#else
    #define LOG(msg)
#endif

int main() {
    int array[MAX_SIZE];
    int result = SQUARE(5); // Expands to ((5) * (5))
    LOG("Starting program");
    return 0;
}
```

Try it: `gcc -E -dM` shows all predefined macros

Stage 2: Compilation

C to Assembly

- Parses preprocessed source
- Performs syntax and semantic analysis
- Optimizes code (if requested)
- Generates assembly language (`.s` file)

Command: `gcc -S source.c -o source.s`

The assembly is human-readable - great for learning!

Stage 3: Assembly

Assembly to Machine Code

- Converts assembly mnemonics to binary opcodes
- Creates object file (`.o` or `.obj`)
- Contains machine code but **not yet executable**
- Symbols are not yet resolved

Command: `gcc -c source.c -o source.o`

Object files are relocatable and can be inspected with `objdump -d`

Stage 4: Linking

Combining Everything

The linker (`ld`) does the hard work:

- Combines multiple object files
- Resolves external symbol references
- Links with libraries (static or shared)
- Resolves addresses and relocations
- Produces final executable or shared library

Command: `gcc source.o -o executable`

Section 1.1: GCC - The GNU Compiler Collection

GCC (GNU Compiler Collection)

More Than Just C

- **C:** gcc
- **C++:** g++
- **Objective-C:** gcc with runtime
- **Fortran:** gfortran
- **Go:** gccgo

For this course, we focus on gcc for C

Essential GCC Options

```
# Basic compilation
gcc source.c -o program

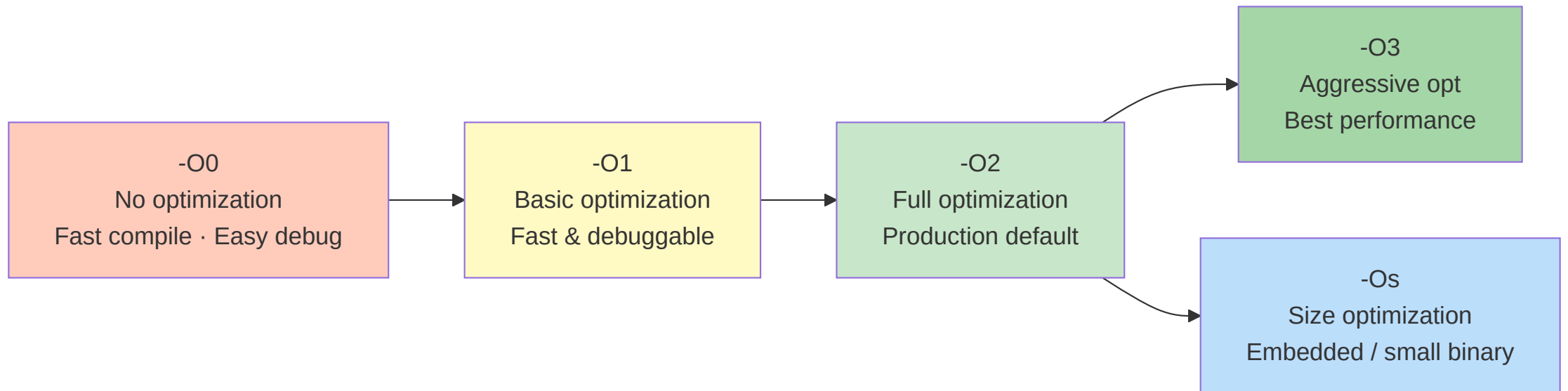
# Include debugging symbols
gcc -g source.c -o program

# Show all warnings
gcc -Wall -Wextra source.c -o program

# Treat warnings as errors
gcc -Wall -Werror source.c -o program

# Specify standard
gcc -std=c11 source.c -o program
```

GCC Optimization Levels



Optimization Flags Explained

Choosing the Right Level

- **-00** (default): No optimization, great for debugging
- **-01** : Basic optimizations, still debuggable
- **-02** : Recommended for production, good balance
- **-03** : Aggressive optimization, may increase code size
- **-0s** : Optimize for size (embedded systems, caching)

Pro tip: Use `-O2` for release builds, `-O0 -g` for debugging

Warning Flags: Your Best Friend

```
# Essential warnings
-Wall           # Enable most warnings
-Wextra        # Extra warnings beyond -Wall
-Werror        # Treat warnings as errors
-Wpedantic     # Strict ISO C compliance

# Specific useful warnings
-Wformat=2     # Format string vulnerabilities
-Wuninitialized # Uninitialized variables
-Wshadow       # Variable shadowing
-Wcast-align   # Pointer cast alignment issues
```

Best practice: Always compile with `-Wall -Wextra` minimum

Alternative Compilers

Clang/LLVM

- Modern alternative to GCC
- Better error messages
- Faster compilation (often)
- Same command-line interface: `clang file.c`

Intel Compiler (icc)

- Optimized for Intel CPUs
- Commercial product
- Often faster binaries on Intel hardware

All modern compilers support mostly the same C standards

Section 2: Libraries

What is a Library?

Code Reuse Mechanism

A library is a collection of precompiled functions/code:

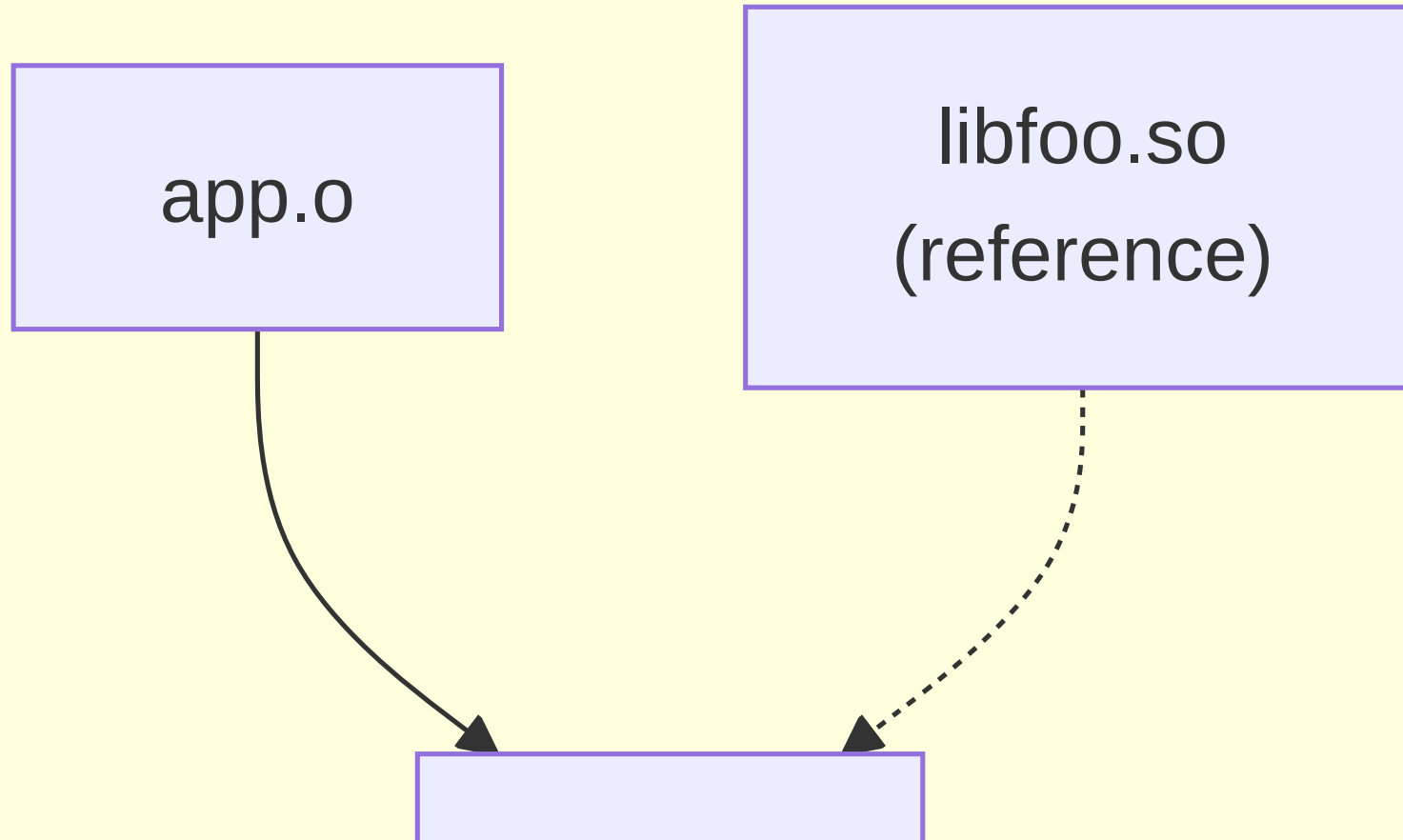
- **Avoid rewriting common functionality**
- **Share code between programs**
- **Easier maintenance and updates**

Two types:

1. **Static libraries** (`.a`): Linked at compile time
2. **Shared libraries** (`.so`): Linked at runtime

Static vs Shared Libraries

Dynamic Linking



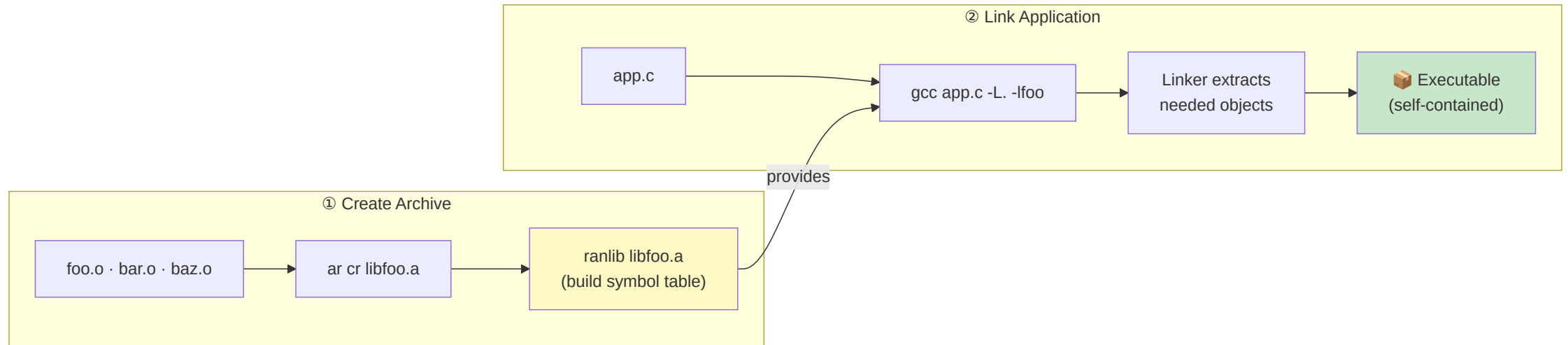
Static Libraries Explained

Archive Files (`.a`)

- Collection of object files bundled together
- Entire library copied into executable
- No runtime dependency
- Larger executable size

Created with: `ar` (archiver tool)

Creating Static Libraries



Static Library Example

```
# 1. Compile source files to object files
gcc -c foo.c -o foo.o
gcc -c bar.c -o bar.o

# 2. Create static library archive
ar rcs libmylib.a foo.o bar.o

# 3. Use the library
gcc main.c -L. -lmylib -o program

# Result: foo.o and bar.o code copied into 'program'
```

Library naming: `lib` prefix + name + `.a` suffix

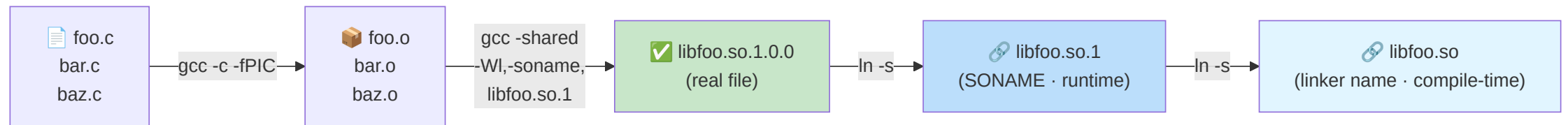
Shared Libraries Explained

Dynamic Link Libraries (`.so`)

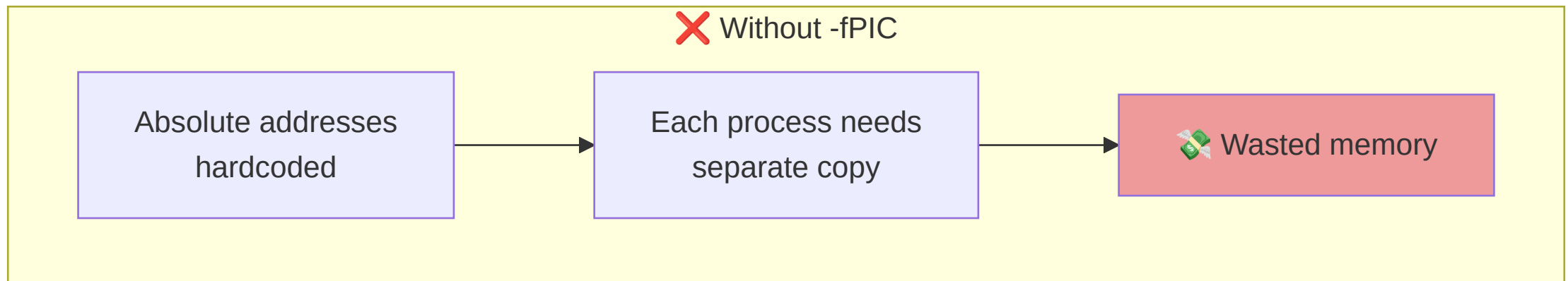
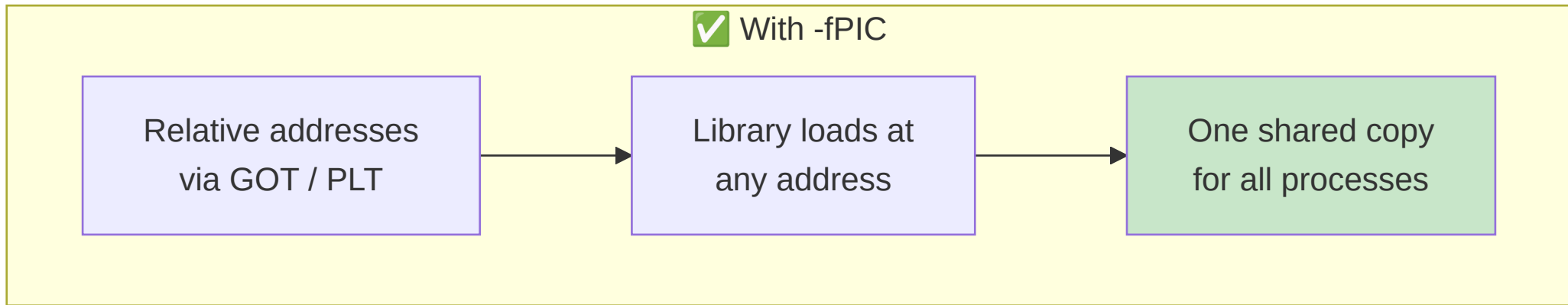
- **Shared Objects** - Linux term for DLLs
- Loaded at runtime by dynamic linker
- Multiple programs share one copy in memory
- Smaller executables
- Can update library without recompiling programs

Trade-off: Runtime dependency management

Creating Shared Libraries



Position Independent Code (PIC)



-fPIC is mandatory for shared libraries

Shared Library Example

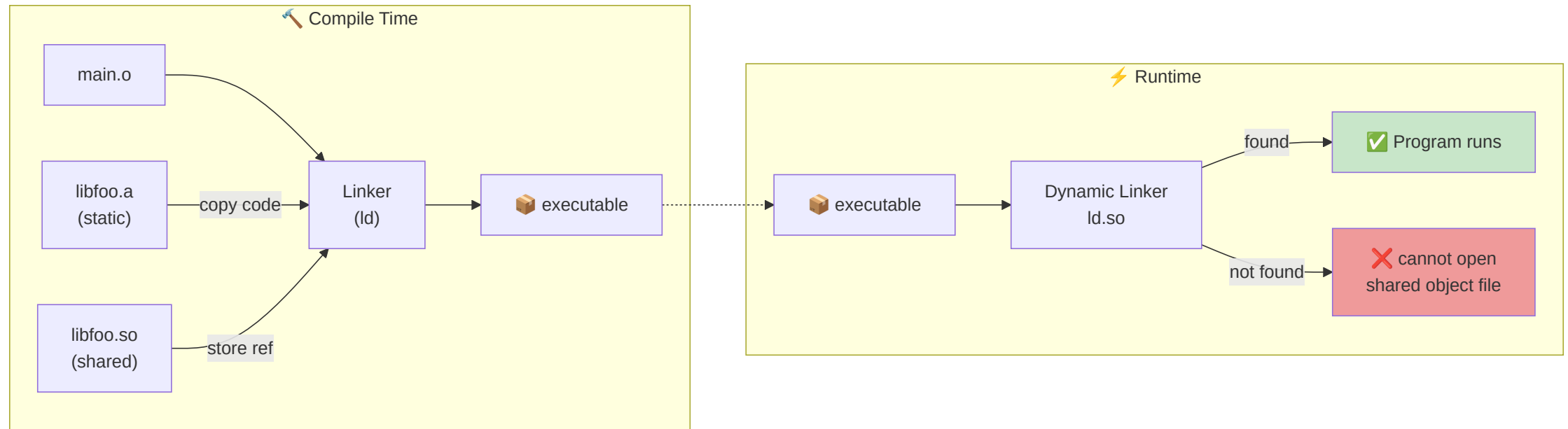
```
# 1. Compile with -fPIC flag
gcc -c -fPIC foo.c -o foo.o
gcc -c -fPIC bar.c -o bar.o

# 2. Create shared library with version
gcc -shared -Wl,-soname,libmylib.so.1 \
    -o libmylib.so.1.0.0 foo.o bar.o

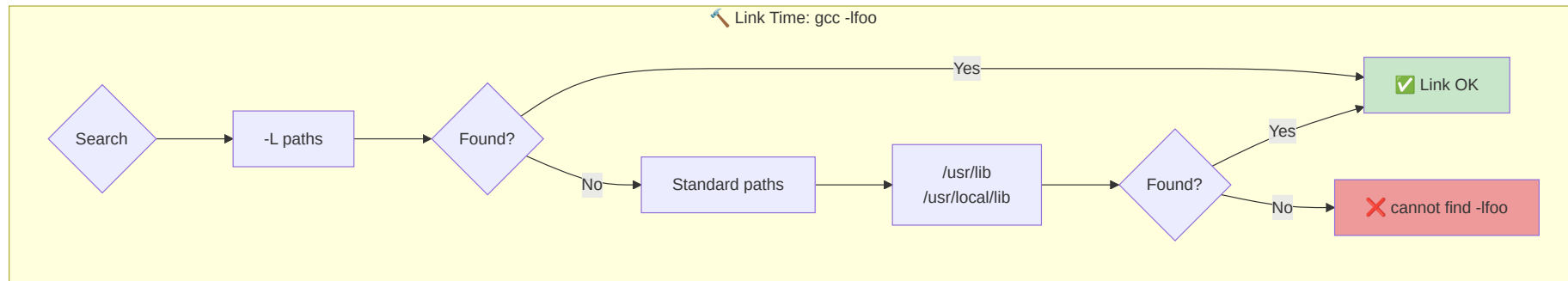
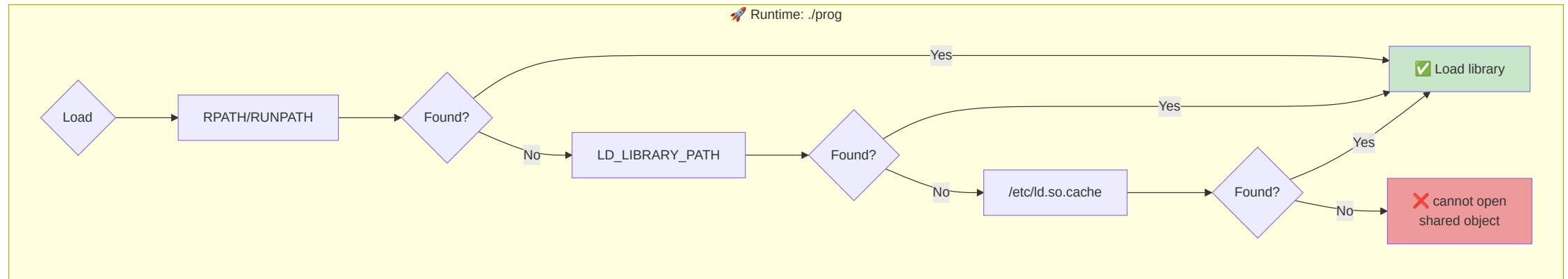
# 3. Create symlinks
ln -s libmylib.so.1.0.0 libmylib.so.1
ln -s libmylib.so.1 libmylib.so

# 4. Use the library
gcc main.c -L. -lmylib -o program
```

Library Linking Process



Library Search Paths



Specifying Library Paths

Compile-time (Linker)

```
# Add library search directory
gcc main.c -L/path/to/libs -lmylib -o program

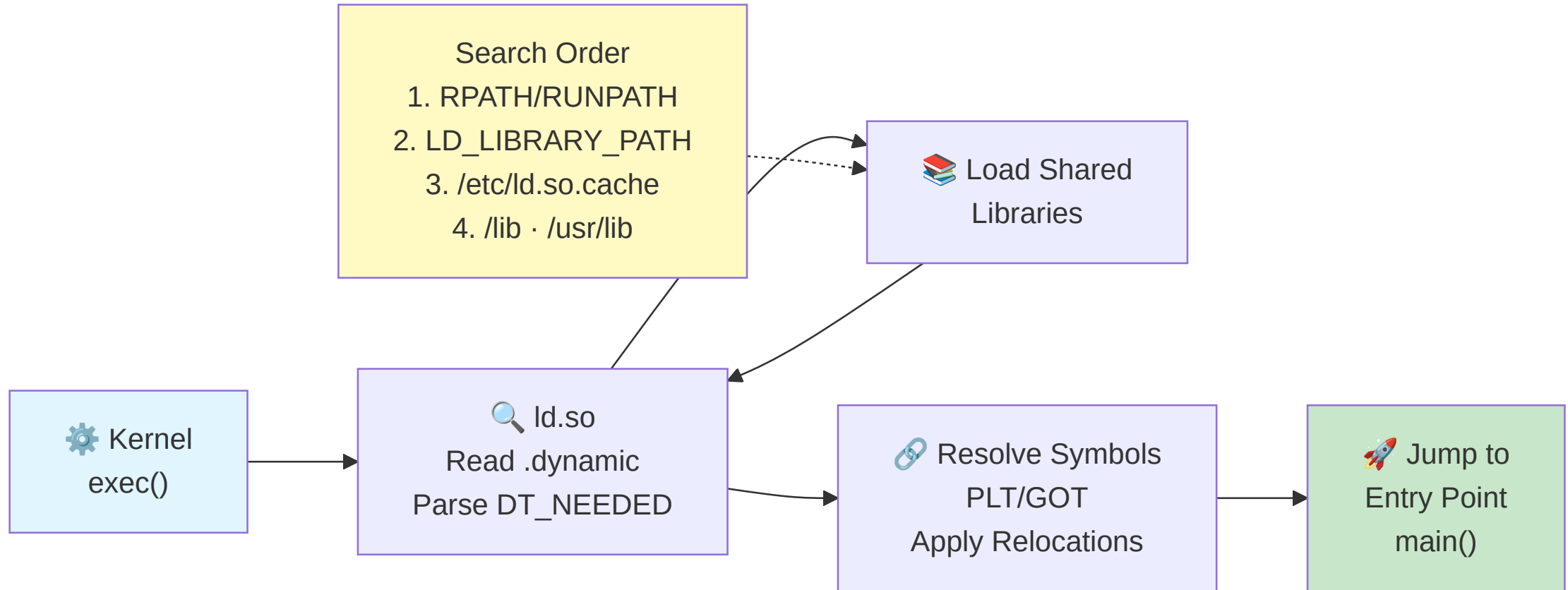
# Multiple paths
gcc main.c -L./lib -L/usr/local/lib -lmylib
```

Runtime (Dynamic Linker)

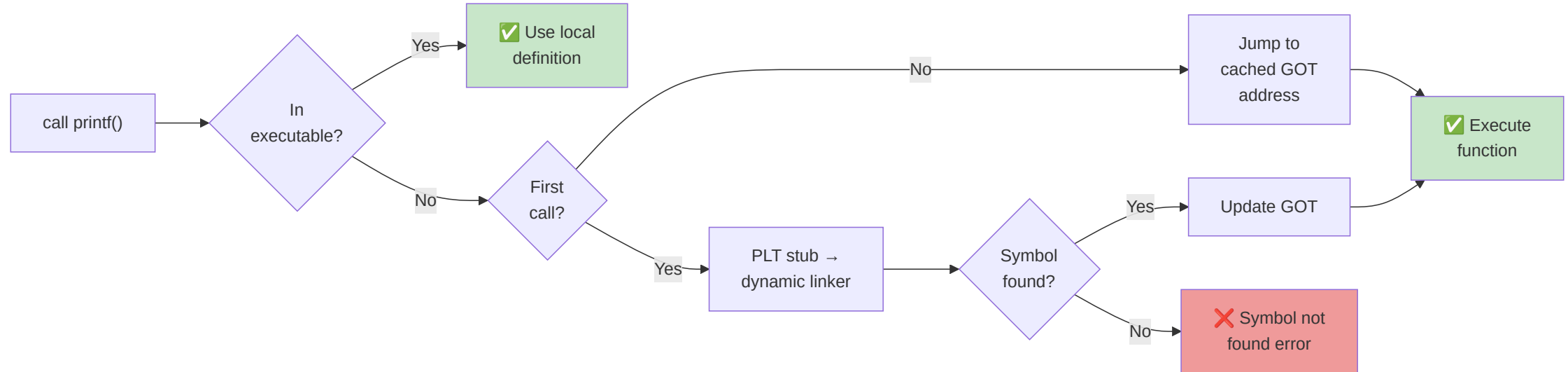
```
# Temporary: Set environment variable
export LD_LIBRARY_PATH=/path/to/libs:$LD_LIBRARY_PATH
./program

# Permanent: Add to ld.so.conf
echo "/path/to/libs" | sudo tee -a /etc/ld.so.conf.d/mylibs.conf
sudo ldconfig
```

The Dynamic Linker/Loader

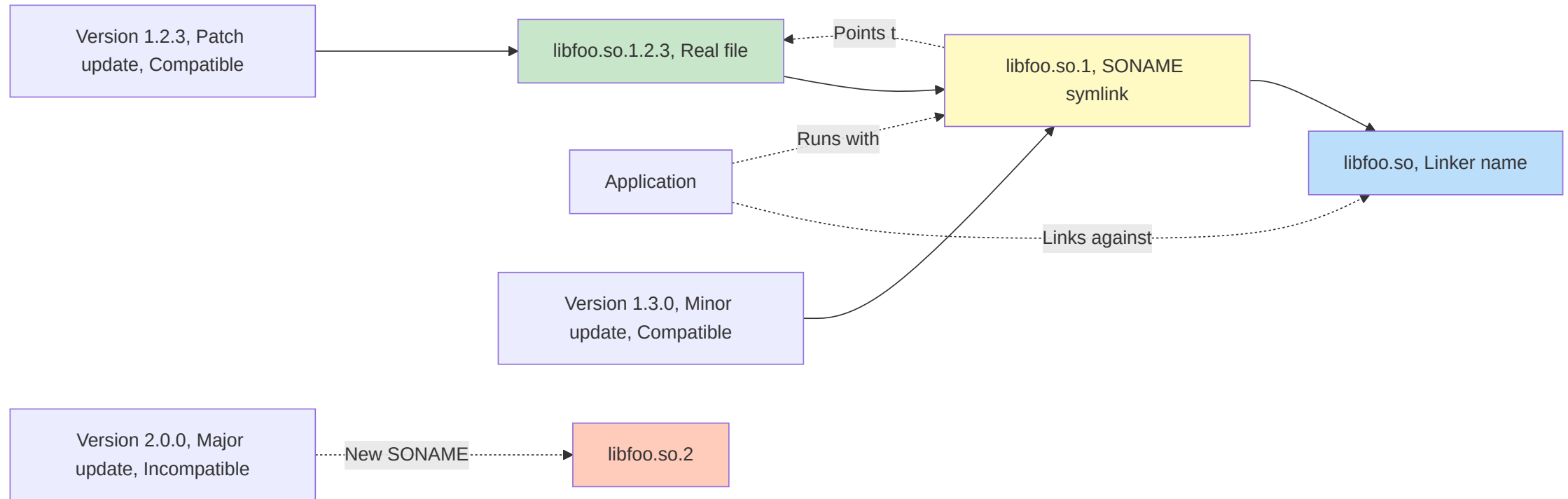


Symbol Resolution



PLT (Procedure Linkage Table) + GOT (Global Offset Table)

Library Versioning with SONAME



SONAME Best Practices

Version Number Format: X.Y.Z

- **X (major):** Incompatible API changes - new SONAME
- **Y (minor):** Backward-compatible additions
- **Z (patch):** Bug fixes only

```
# Version 1.2.3 of libfoo
libfoo.so.1.2.3      # Real file
libfoo.so.1         # SONAME (major version only)
libfoo.so            # Linker name (development)

# Breaking change → bump major version
libfoo.so.2.0.0      # New real file
libfoo.so.2          # New SONAME
```

Applications depend on SONAME, not specific patch version

Essential Library Tools

```
# List library dependencies
ldd program

# Inspect library symbols
nm libmylib.a
nm -D libmylib.so

# Check library SONAME
objdump -p libmylib.so | grep SONAME

# Update dynamic linker cache
sudo ldconfig

# List cached libraries
ldconfig -p | grep libmylib
```

Common Library Issues

"cannot open shared object file"

```
# Problem: Library not found at runtime
./program: error while loading shared libraries:
  libmylib.so.1: cannot open shared object file

# Solutions:
# 1. Add to LD_LIBRARY_PATH
export LD_LIBRARY_PATH=/path/to/lib:$LD_LIBRARY_PATH

# 2. Install library systemwide
sudo cp libmylib.so.1.0.0 /usr/local/lib/
sudo ldconfig

# 3. Use RPATH (embedded in executable)
gcc -Wl,-rpath,/path/to/lib main.c -lmylib
```

Section 3: Build Systems with Make

Why Build Systems?

Problems Make Solves

- **Avoid recompiling unchanged files** - faster builds
- **Manage dependencies** - recompile only what's needed
- **Automate repetitive tasks** - consistency
- **Complex multi-file projects** - track relationships

Make is the classic Unix build system

Makefile Basics

Automatic Variables

`$@`: Target name

`$<`: First prerequisite

`^`: All prerequisites

Variables

`CC = gcc`

`OBJECTS = main.o utils.o`

`CFLAGS = -Wall -O2`

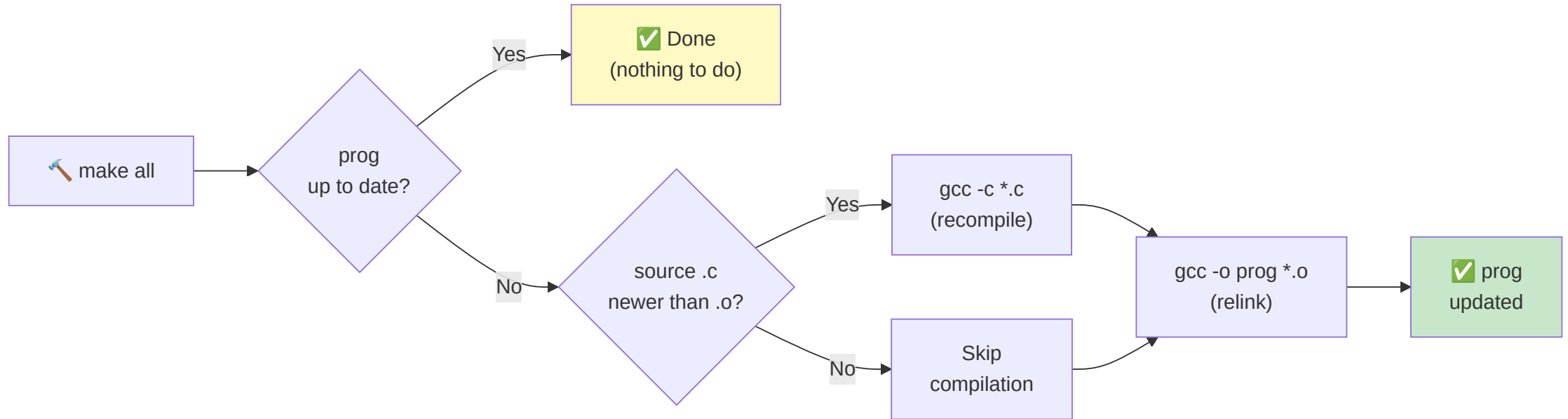
Makefile Anatomy

Target: `prog`

Prerequisites: `main.o utils.o`

Recipe: `gcc -o prog main.o
utils.o`

How Make Works



Make compares timestamps - only rebuilds if source is newer

Your First Makefile

```
# Simple Makefile
program: main.o utils.o
    gcc -o program main.o utils.o

main.o: main.c utils.h
    gcc -c main.c

utils.o: utils.c utils.h
    gcc -c utils.c

clean:
    rm -f program *.o
```

Indentation MUST be tabs, not spaces!

Makefile Variables

```
# Define variables
CC = gcc
CFLAGS = -Wall -Wextra -g -O2
LDFLAGS = -lm
OBJECTS = main.o utils.o calc.o

# Use variables
program: $(OBJECTS)
    $(CC) -o program $(OBJECTS) $(LDFLAGS)

%.o: %.c
    $(CC) $(CFLAGS) -c $< -o $@
```

Variables make Makefiles maintainable and portable

Automatic Variables in Make

Special Built-in Variables

```
$@ # Target name
$< # First prerequisite
$^ # All prerequisites
$? # Prerequisites newer than target
$* # Stem of pattern match

# Example usage
%.o: %.c
    $(CC) $(CFLAGS) -c $< -o $@
    # $< = source.c, $@ = source.o
```

These save typing and make rules more generic

Pattern Rules

```
# Generic rule for all .o files from .c files
%.o: %.c
    $(CC) $(CFLAGS) -c $< -o $@

# Multiple patterns
%: %.c
    $(CC) $(CFLAGS) $< -o $@

# With static patterns
OBJECTS = main.o utils.o math.o
$(OBJECTS): %.o: %.c
    $(CC) $(CFLAGS) -c $< -o $@
```

Pattern rules eliminate repetition

Phony Targets

```
# Declare targets that aren't files
.PHONY: all clean install test

all: program

clean:
    rm -f program *.o

install: program
    cp program /usr/local/bin/

test: program
    ./run_tests.sh
```

Without `.PHONY`, Make would look for files named "clean", "all", etc.

Sophisticated Makefile Example

```
CC = gcc
CFLAGS = -Wall -Wextra -std=c11 -O2
LDFLAGS = -lm -lpthread
TARGET = myapp
SOURCES = $(wildcard src/*.c)
OBJECTS = $(SOURCES:.c=.o)
DEPS = $(OBJECTS:.o=.d)

# Main target
$(TARGET): $(OBJECTS)
    $(CC) -o $@ $^ $(LDFLAGS)

# Include dependencies
-include $(DEPS)

# Generate dependencies automatically
%.o: %.c
    $(CC) $(CFLAGS) -MMD -c $< -o $@

.PHONY: clean
clean:
    rm -f $(TARGET) $(OBJECTS) $(DEPS)
```

Building Large Projects

Multi-directory Structure

```
project/
├── Makefile           # Top-level
├── src/
│   ├── Makefile
│   └── *.c
├── lib/
│   ├── Makefile
│   └── *.c
└── include/
    └── *.h
```

Recursive Make

```
SUBDIRS = src lib
```

```
all:
```

```
    for dir in $(SUBDIRS); do \  
        $(MAKE) -C $$dir; \  
    done
```

Make Best Practices

Do's

- ✓ Use variables for compiler settings
- ✓ Declare `.PHONY` targets
- ✓ Auto-generate dependencies (`-MMD -MP`)
- ✓ Use pattern rules for repetitive tasks
- ✓ Parallel builds: `make -j$(nproc)`

Don'ts

- ✗ Use spaces instead of tabs
- ✗ Hardcode paths and filenames
- ✗ Forget to handle errors
- ✗ Mix tabs and spaces

Alternative Build Systems

CMake

- Cross-platform build configuration
- Generates Makefiles or IDE project files
- Better for large projects

Ninja

- Fast, parallel-by-default
- Often used with CMake

Meson

- Modern, Python-based
- Fast and user-friendly

For this course, we focus on Make as the foundation

Section 4: Version Control with Git

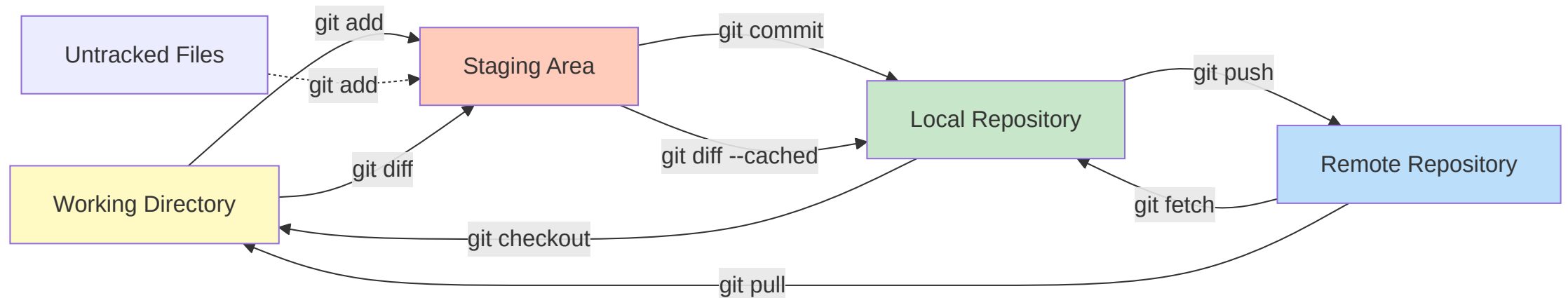
Why Version Control?

Essential for Professional Development

- **Track changes over time** - who changed what, when, why
- **Collaborate with team** - merge changes safely
- **Experiment safely** - branches for features
- **Rollback mistakes** - recover old versions
- **Code review** - track what's being changed

Git is the industry standard (96%+ adoption)

Git Workflow Fundamentals



Git Basic Operations

```
# Initialize repository
```

```
git init
```

```
# Add files to staging
```

```
git add file.c
```

```
git add *.c
```

```
# Commit changes
```

```
git commit -m "Add feature X"
```

```
# See status
```

```
git status
```

```
# View history
```

```
git log --oneline --graph
```

```
# See changes
```

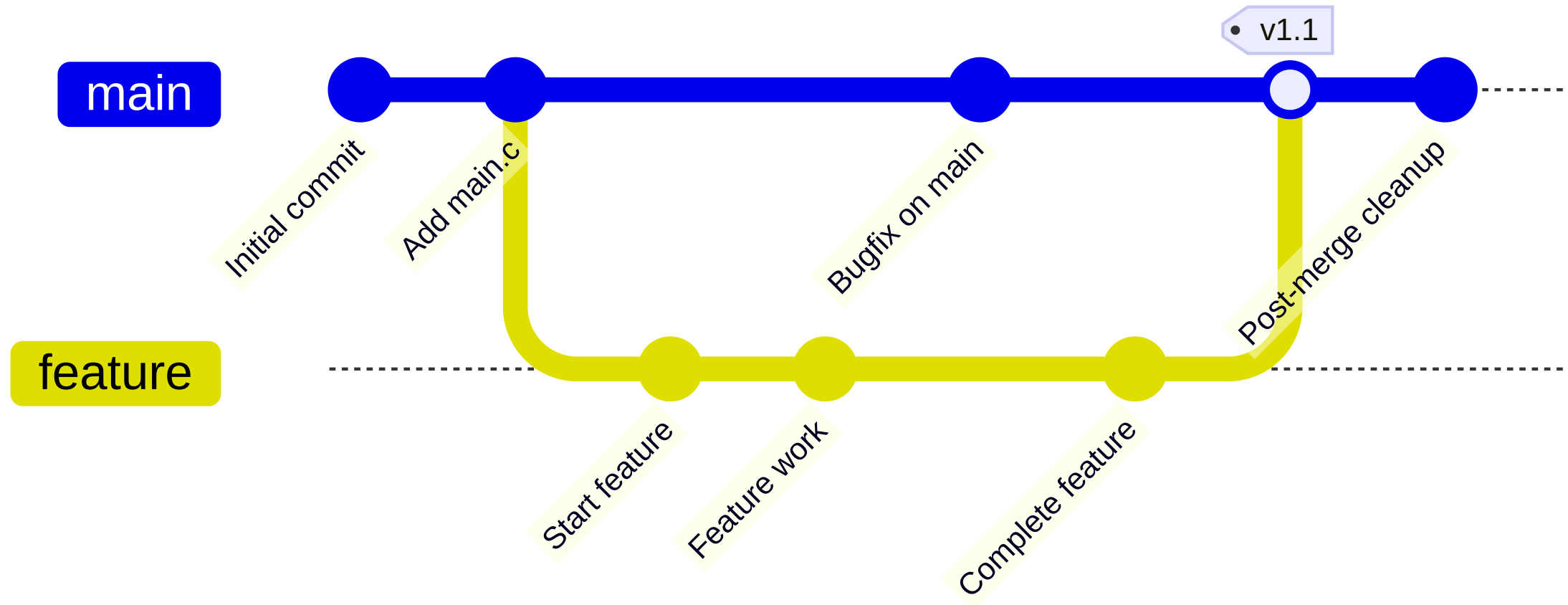
```
git diff
```

```
# Working vs staging
```

```
git diff --cached
```

```
# Staging vs repository
```

Git Branching



Branching Commands

```
# Create branch
git branch feature-x

# Switch to branch
git checkout feature-x
# Or in one command
git checkout -b feature-x

# List branches
git branch -a

# Merge branch
git checkout main
git merge feature-x

# Delete branch
git branch -d feature-x
```

Remote Repositories

```
# Clone repository
git clone https://github.com/user/repo.git

# Add remote
git remote add origin https://github.com/user/repo.git

# Push changes
git push origin main

# Pull changes
git pull origin main

# Fetch without merging
git fetch origin
```

Git for C Projects

Essential Files

.gitignore - Don't track generated files

```
# Object files
*.o
*.a
*.so
*.so.*

# Executables
*.out
a.out
program

# Build directories
build/
dist/
```

Commit Best Practices

Good Commit Messages

```
# Good: Descriptive and specific  
git commit -m "Fix memory leak in parser_init()"  
git commit -m "Add error handling to file I/O functions"
```

```
# Bad: Vague and unhelpful  
git commit -m "fix bug"  
git commit -m "changes"  
git commit -m "work in progress"
```

Commit Guidelines

- One logical change per commit
- Write clear, descriptive messages
- Test before committing
- Commit related changes together

Advanced Git Operations

```
# Undo last commit (keep changes)
git reset --soft HEAD~1

# Discard all local changes
git reset --hard HEAD

# Create annotated tag
git tag -a v1.0 -m "Version 1.0 release"

# Cherry-pick single commit
git cherry-pick abc123

# Stash changes temporarily
git stash
git stash pop

# Rebase instead of merge
git rebase main
```

Git Workflows for Teams

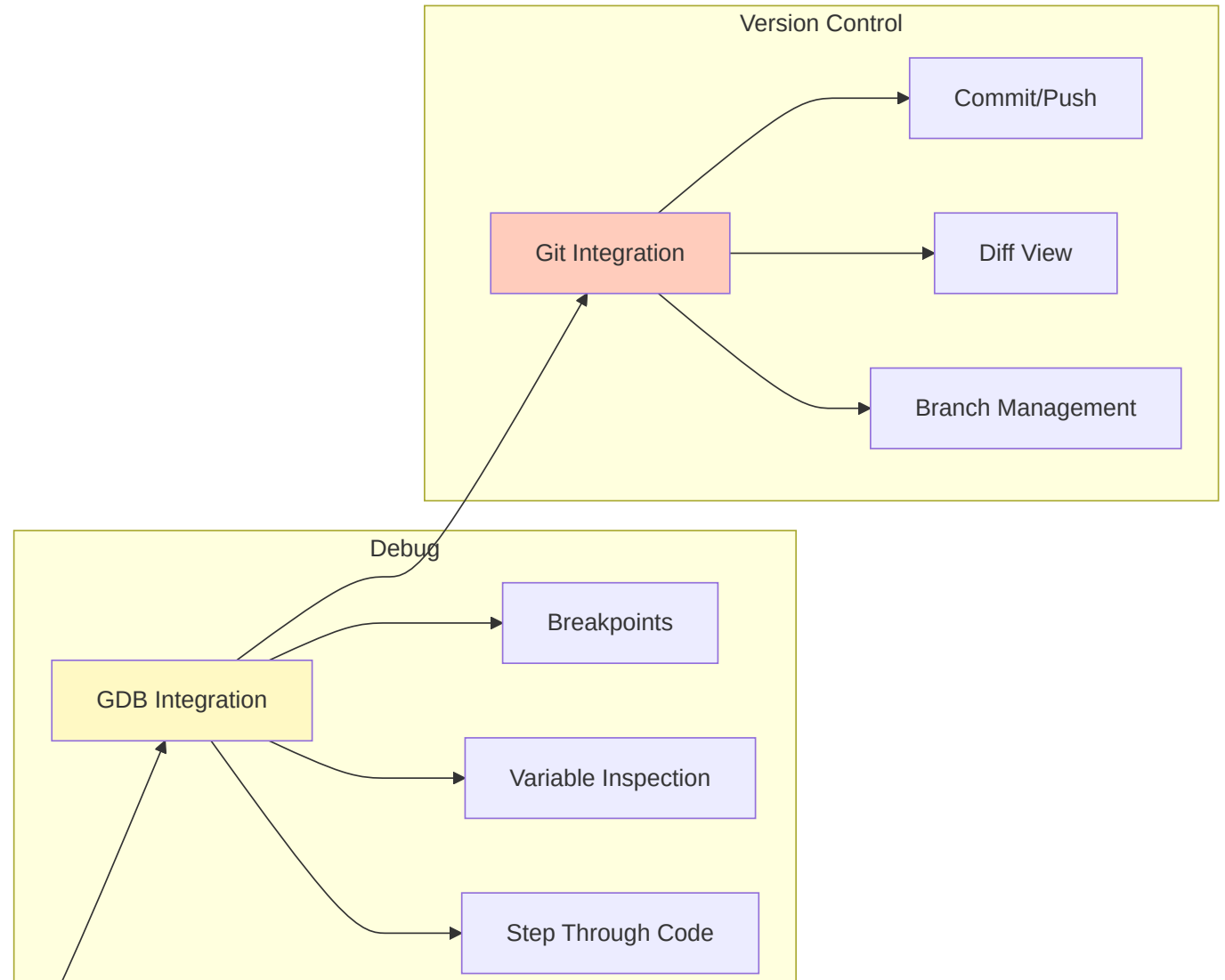
Feature Branch Workflow

1. Create branch from `main`
2. Develop feature
3. Push branch to remote
4. Create pull request
5. Code review
6. Merge to `main`
7. Delete feature branch

Keeps `main` always deployable

Section 5: Development Environments

IDE Features for C Development



Popular IDEs for Linux Development

Visual Studio Code

- Lightweight, extensible
- C/C++ extension by Microsoft
- Integrated debugging
- **Recommended for this course**

CLion

- Professional IDE by JetBrains
- Advanced refactoring
- Commercial (free for students)

Eclipse CDT

- Full-featured, open-source
- Heavy on resources
- Good for large projects

VS Code Setup for C

Essential Extensions

```
# Install C/C++ extension
code --install-extension ms-vscode.cpptools

# Install Makefile tools
code --install-extension ms-vscode.makefile-tools

# Install GitLens (Git supercharged)
code --install-extension eamodio.gitlens
```

Configure compiler

```
{  
  "C_Cpp.default.compilerPath": "/usr/bin/gcc",  
  "C_Cpp.default.cStandard": "c11",  
  "C_Cpp.default.intelliSenseMode": "linux-gcc-x64"  
}
```

Command-Line Power Tools

Essential for Linux Developers

```
# Fast file search
find . -name "*.c"

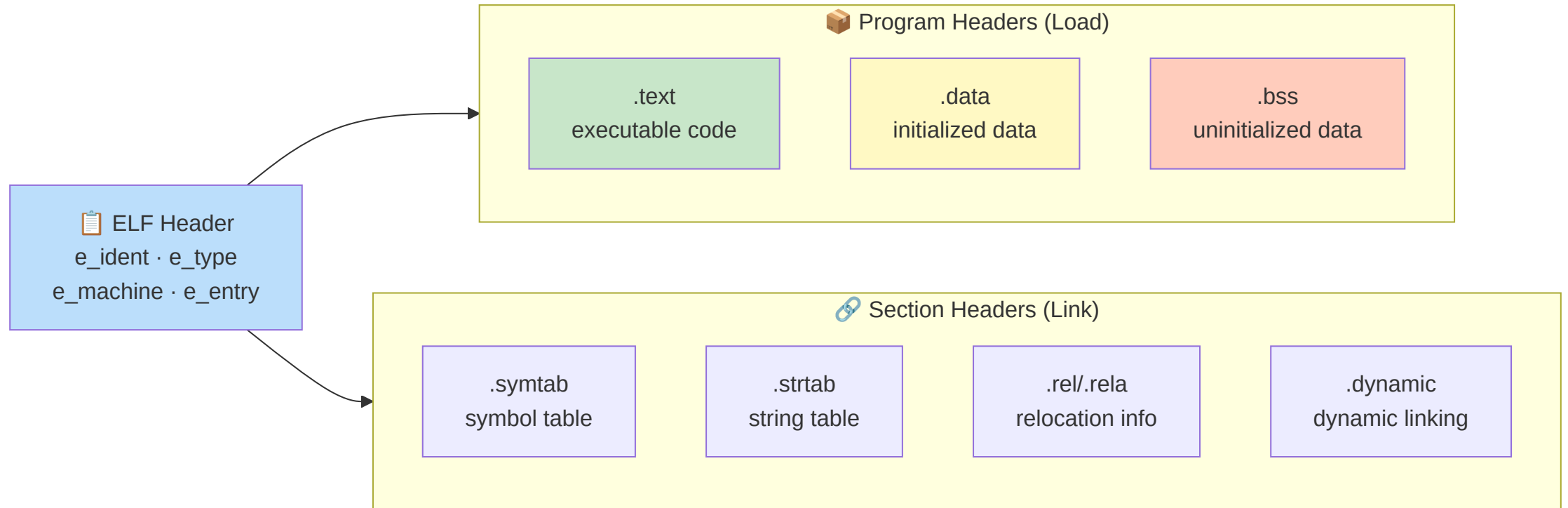
# Content search
grep -r "function_name" src/

# Code patterns
ag "TODO|FIXME" --cc # Silver Searcher

# Navigate code
ctags -R . # Generate tag file
vim -t function_name # Jump to definition
```

IDE + command-line tools = maximum productivity

ELF File Format Deep Dive



Understanding ELF helps debug linking problems

Inspecting Binaries

```
# View ELF header  
readelf -h program
```

```
# View section headers  
readelf -S program
```

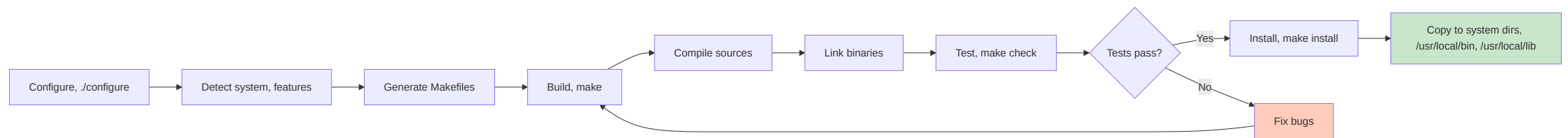
```
# View symbol table  
nm program  
readelf -s program
```

```
# View dynamic dependencies  
ldd program
```

```
# Disassemble  
objdump -d program  
objdump -S program # With source (if -g used)
```

```
# Strings in binary  
strings program
```

Build System Automation



Day 1 Summary

What We Covered

- ✓ **Compilation pipeline** - 4 stages from source to binary
- ✓ **GCC essentials** - Options, optimization, warnings
- ✓ **Static libraries** - Creating and using `.a` archives
- ✓ **Shared libraries** - PIC, SONAME, dynamic linking
- ✓ **Make build system** - Makefiles, variables, patterns
- ✓ **Git version control** - Workflow, branching, collaboration
- ✓ **Development tools** - IDEs and inspection tools

Key Takeaways

Critical Concepts

1. Always use `-Wall -Wextra` when compiling
2. Shared libraries reduce memory but add complexity
3. SONAME versioning prevents breakage
4. Make tracks dependencies for efficient rebuilds
5. Git branches enable safe experimentation
6. Know your toolchain - inspect binaries when debugging

Common Pitfalls to Avoid

- ✗ Forgetting `-fPIC` for shared libraries
- ✗ Not setting `LD_LIBRARY_PATH` for custom libraries
- ✗ Using spaces instead of tabs in Makefiles
- ✗ Committing generated files (`.o`, binaries) to Git
- ✗ Ignoring compiler warnings
- ✗ Optimizing before code is correct

Debug with `-O0 -g`, release with `-O2 -Wall -Werror`

Hands-On Exercises

Lab Time: ~4 Hours

Navigate to: `exercises/01-build-tools-debugging-setup.md`

You will:

1. Compile multi-stage programs manually
2. Create static and shared libraries
3. Write Makefiles for real projects
4. Set up Git repository for C project
5. Debug library linking issues
6. Inspect binaries with tools

Tomorrow: Day 2

Debugging and System Fundamentals

- GDB debugging techniques
- Valgrind memory analysis
- System calls and kernel interface
- Memory management internals
- Performance profiling with perf

Exercises connect: Use Make to build programs we'll debug tomorrow

Questions?

Resources

- GCC Manual: `info gcc`
- Make Manual: `info make`
- Git Book: `git-scm.com/book`
- Man pages: `man gcc` , `man ld` , `man make`

Office hours: After lab session

Additional Resources

Online Documentation

- [GCC Online Docs](#)
- [Make Manual](#)
- [Pro Git Book](#)
- [ELF Format](#)

Tools Reference

- `man gcc` , `man ld` , `man ar` , `man nm`
- `info make`
- `git help <command>`

Break!

Next Up: Hands-On Lab

 Take 10 minutes, then dive into the exercises

Remember: The best way to learn is by doing!